Boolean Satisfiability

Edward A. Hirsch

http://edwardahirsch.github.io/edwardahirsch

Ariel University

January 1, 2024

Course Logistics

- The Boolean satisfiability problem is a central problem in TCS.
- Lots of industrial (and also scientific) applications of SAT algorithms and UNSAT proofs.
- Course overview coming after I remind the definitions (30 min).

Course Logistics

- The Boolean satisfiability problem is a central problem in TCS.
- Lots of industrial (and also scientific) applications of SAT algorithms and UNSAT proofs.
- Course overview coming after I remind the definitions (30 min).

- Three hours a week: Monday 15:00–18:00, room 20.2.12.
- It is strictly prohibited to share course materials that are not published openly (including your own recordings, if you record). You can share with your fellow participants of this year run only.

Course Logistics

- The Boolean satisfiability problem is a central problem in TCS.
- Lots of industrial (and also scientific) applications of SAT algorithms and UNSAT proofs.
- Course overview coming after I remind the definitions (30 min).

- Three hours a week: Monday 15:00–18:00, room 20.2.12.
- It is strictly prohibited to share course materials that are not published openly (including your own recordings, if you record). You can share with your fellow participants of this year run only.

- Two assignments (written homeworks), 40% each.
- Final exam, 20%.

Course Logistics

- The Boolean satisfiability problem is a central problem in TCS.
- Lots of industrial (and also scientific) applications of SAT algorithms and UNSAT proofs.
- Course overview coming after I remind the definitions (30 min).

- Three hours a week: Monday 15:00–18:00, room 20.2.12.
- It is strictly prohibited to share course materials that are not published openly (including your own recordings, if you record). You can share with your fellow participants of this year run only.

- Two assignments (written homeworks), 40% each.
- Final exam, 20%.

- Course materials will be later posted at:
  https://edwardahirsch.github.io/edwardahirsch/courses.html
- Have a question?
  - During lecture — ASK! (not in the middle of a statement ☺).
  - Between the lectures — write me an email and receive my answer (or schedule a meeting).

The Rules

You better find them in the syllabus!

Allowed material for the final exam (20% ☺):

1. A physical English dictionary (translating to any language you wish).
2. Your own handwritten (yes, written by your own hand; printed hand-written text that was prepared using a tablet is permitted) physical (not electronic) lecture notes of reasonable size (yes, they can cover the whole contents of the lectures). Be ready to present them before or during the exam if you are asked to do so (still you can use any language for your notes).

Any electronic equipment (including headsets and digital watches) is strictly prohibited, you have to check that you left all the electronics in your bag and turned off the sound and vibration completely.

Assignments (40% + 40%):
This is your assignment, you complete it yourself. It is allowed to consult other students, literature, or the internet, but it should not lead to identical write-ups! You must understand well what you write (according to the university policy you can be called for a personal interview
no worries if you wrote it yourself, it's a formality ☺).

I am here to help you ☺

- Draft slides will come before a lecture.
- Corrected slides and lecture notes will come before the next lecture.
  - Make your own notes anyway for
    (a) completeness;
    (b) final exam.
- We will see today how the recording equipment works...
- Assignment problems: one by one, presented during lectures.
- Office hours: I am happy ☺ to meet you
  - in person,
  - on zoom,
  - by email.

  In any case, send me an email to edward.a.hirsch@gmail.com !
  Do not delay it!

Terminology: Boolean Satisfiability

Formulas

- Boolean (propositional) variables $x_1, x_2, \ldots, x_n$ that can take (truth) values `True` or `False` (aka `1` or `0`).
- Literals are variables x_i ("positive literals") or negations of variables $\overline{x_i}$ ("negative literals").
- Clauses are sets of literals interpreted as disjunctions $\ell_1 \vee \ldots \vee \ell_k$. Normally, we assume that x and $\overline{x}$ do not occur together.

Terminology: Boolean Satisfiability

Formulas

- Boolean (propositional) variables $x_1, x_2, \ldots, x_n$ that can take (truth) values `True` or `False` (aka `1` or `0`).
- Literals are variables x_i ("positive literals") or negations of variables $\overline{x_i}$ ("negative literals").
- Clauses are sets of literals interpreted as disjunctions $\ell_1 \vee \ldots \vee \ell_k$. Normally, we assume that x and $\overline{x}$ do not occur together.
- Formula in conjunctive normal form (CNF) is a conjunction of clauses (maxterms): $F = C_1 \wedge \ldots \wedge C_m$.

$$(x \vee y) \wedge (\overline{x} \vee \overline{y}) \wedge (x \vee \overline{y} \vee z) \wedge (\overline{x} \vee y)$$

Terminology: Boolean Satisfiability

Formulas

- Boolean (propositional) variables $x_1, x_2, \ldots, x_n$ that can take (truth) values `True` or `False` (aka `1` or `0`).
- Literals are variables x_i ("positive literals") or negations of variables $\overline{x_i}$ ("negative literals").
- Clauses are sets of literals interpreted as disjunctions $\ell_1 \vee \ldots \vee \ell_k$. Normally, we assume that x and $\overline{x}$ do not occur together.
- Formula in conjunctive normal form (CNF) is a conjunction of clauses (maxterms): $F = C_1 \wedge \ldots \wedge C_m$.

$$(x \vee y) \wedge (\overline{x} \vee \overline{y}) \wedge (x \vee \overline{y} \vee z) \wedge (\overline{x} \vee y)$$

- Formula in disjunctive normal form (DNF) is a disjunction of conjunctions (minterms):

$$(\overline{x} \wedge \overline{y}) \vee (x \wedge y) \vee (\overline{x} \wedge y \wedge \overline{z}) \vee (x \wedge \overline{y})$$

Terminology: Boolean Satisfiability

Truth Assignments

- A (complete) (truth) assignment maps all variables to truth values.
- A partial assignment maps some variables to their values.

assignment $x \leftarrow \texttt{False},\ y \leftarrow \texttt{True},\ z \leftarrow \texttt{True}$
assignment $\overline{x}, y, z$

Terminology: Boolean Satisfiability

Truth Assignments

- A (complete) (truth) assignment maps all variables to truth values.
- A partial assignment maps some variables to their values.

assignment $x \leftarrow \texttt{False},\ y \leftarrow \texttt{True},\ z \leftarrow \texttt{True}$
assignment $\overline{x}, y, z$

- Substitution $F[A]$:
  - remove all the satisfied clauses,
  - remove false literals from the other clauses.

$$
\begin{aligned}
F &= (x \vee y) \wedge (\overline{x} \vee \overline{y}) \wedge (x \vee \overline{y} \vee z) \wedge (\overline{x} \vee y), \\
F[x \leftarrow \texttt{True}] &= \cancel{(x \vee y)} \wedge (\cancel{\overline{x}} \vee \overline{y}) \wedge (\overline{y} \vee z) \wedge (\cancel{\overline{x}} \vee y)
\end{aligned}
$$

Terminology: Boolean Satisfiability

Truth Assignments

- A (complete) (truth) assignment maps all variables to truth values.
- A partial assignment maps some variables to their values.

assignment $x \leftarrow \texttt{False},\ y \leftarrow \texttt{True},\ z \leftarrow \texttt{True}$
assignment $\overline{x}, y, z$

- Substitution $F[A]$:
  - remove all the satisfied clauses,
  - remove false literals from the other clauses.

$$
\begin{aligned}
F &= (x \vee y) \wedge (\overline{x} \vee \overline{y}) \wedge (x \vee \overline{y} \vee z) \wedge (\overline{x} \vee y), \\
F[x \leftarrow \texttt{True}] &= \cancel{(x \vee y)} \wedge (\cancel{\overline{x}} \vee \overline{y}) \wedge (\overline{y} \vee z) \wedge (\cancel{\overline{x}} \vee y)
\end{aligned}
$$

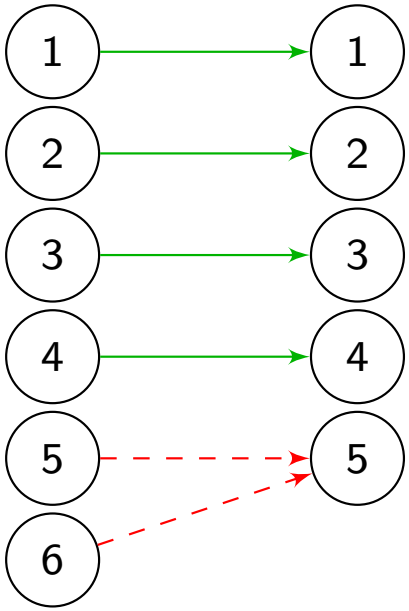
CNF

- A satisfying assignment A satisfies all clauses, that is, $F[A] = \texttt{True}$.
- If a formula has at least one satisfying assignment, it is satisfiable; otherwise it is unsatisfiable.

PHP: the propositional pigeonhole principle

(Important) example of an unsatisfiable formula:

- $x_{i,j} \sim$ pigeon i sits in hole j
  $(1 \leqslant i \leqslant m,\ 1 \leqslant j \leqslant m-1)$.
- $\overline{x_{i,j}} \vee \overline{x_{i',j}}$ (for each $i, j, i' \neq i$):
  two pigeons i, i' do not share the same hole j,
- $x_{i,1} \vee \ldots \vee x_{i,m-1}$ (for each i):
  pigeon i is assigned some hole.



The (unsatisfiable) negation of PHP is in CNF.
The tautology would be in DNF.

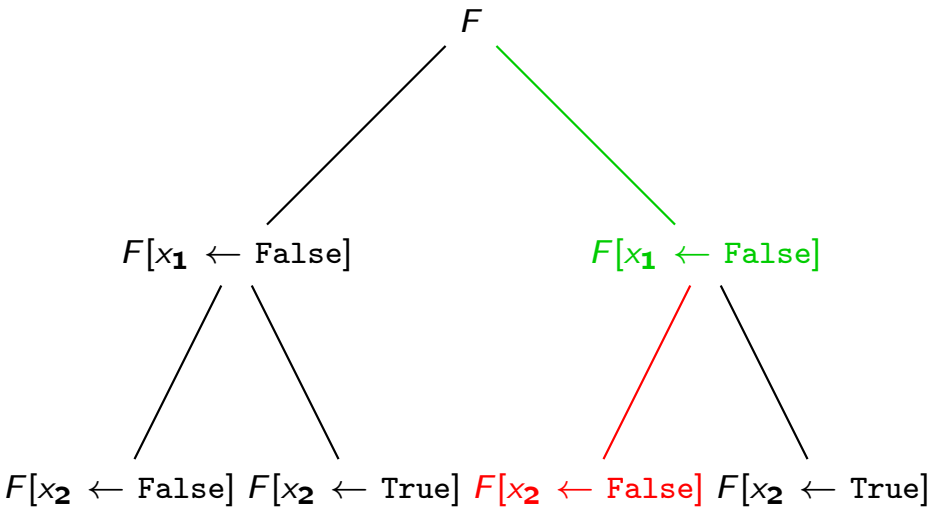
Terminology: Satisfiability as a problem (SAT)

Search or Decision?

- **SAT**: Given a formula, check its satisfiability (`True`/`False`).
- Or: Given a formula, find a satisfying assignment (or say "no").
- `Search-to-decision(`F`):`

```
    if (not SAT(F)) then print("no")
    elif (SAT(F[x ← 0]))            // x is the first variable
      then { print(x̄); Search-to-decision(F[x ← 0]); }
      else { print(x); Search-to-decision(F[x ← 1]); }
```

You only have to process a single path!



Terminology: Satisfiability as a problem (SAT)

- **SAT** is in **NP** as it is easy to verify candidate solutions.
- **SAT** is **NP**-complete (Cook–Levin Theorem), that is, other problems in **NP** reduce to it in polynomial time.
- Moreover, they reduce in a natural way, which is very useful.

Terminology: Satisfiability as a problem (SAT)

- **SAT** is in **NP** as it is easy to verify candidate solutions.
- **SAT** is **NP**-complete (Cook–Levin Theorem), that is, other problems in **NP** reduce to it in polynomial time.
- Moreover, they reduce in a natural way, which is very useful.
- k-CNF: every clause contains at most k literals; the problem: ***k*-SAT**.
  A clause containing k literals: k-clause.
- Even **3-SAT** is already **NP**-complete.

Terminology: Satisfiability as a problem (SAT)

- **SAT** is in **NP** as it is easy to verify candidate solutions.
- **SAT** is **NP**-complete (Cook–Levin Theorem), that is, other problems in **NP** reduce to it in polynomial time.
- Moreover, they reduce in a natural way, which is very useful.
- k-CNF: every clause contains at most k literals; the problem: ***k*-SAT**. A clause containing k literals: k-clause.
- Even **3-SAT** is already **NP**-complete.
- Sometimes **SAT** is called "CNF SAT". One can consider non-CNF SAT (arbitrary formulas using Boolean operations).

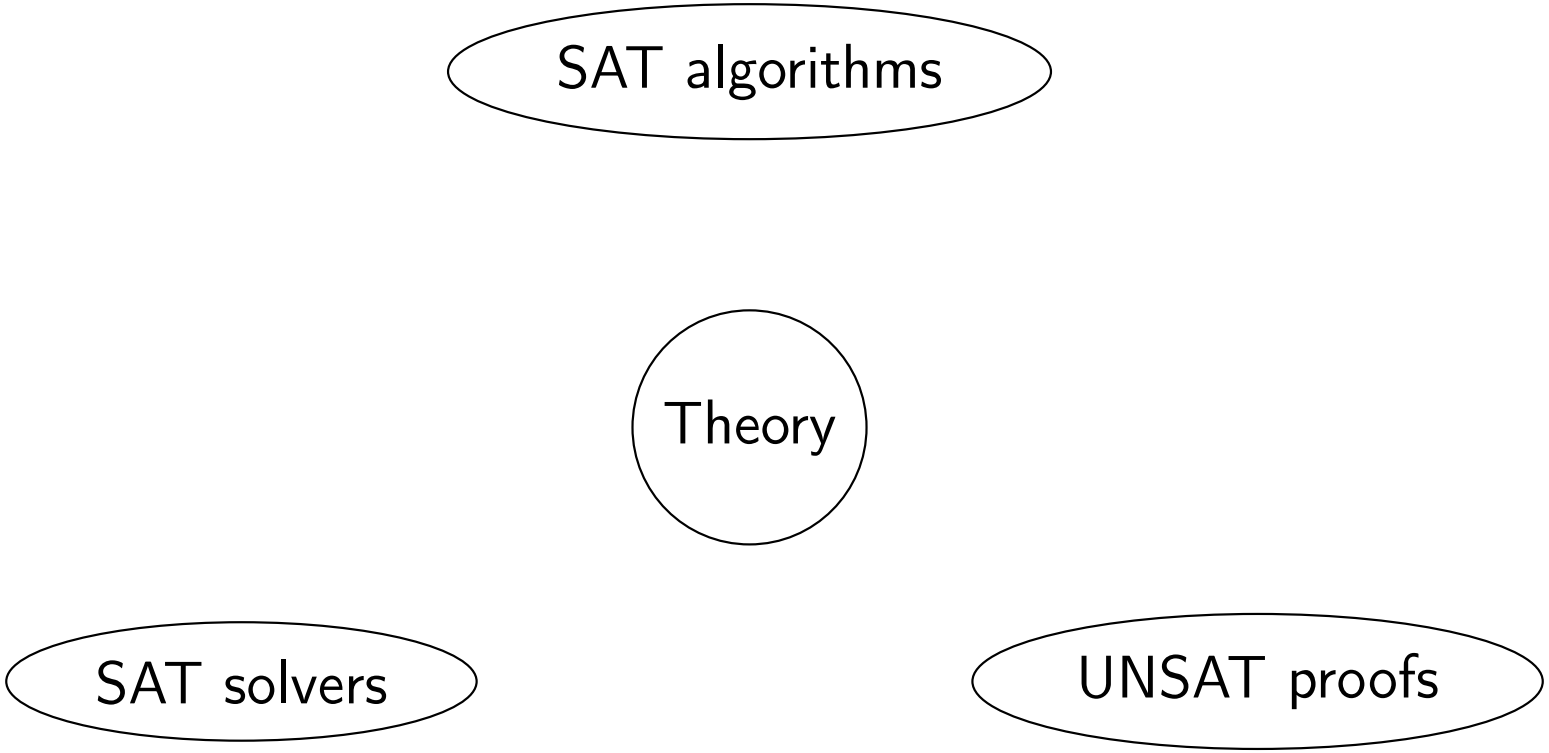
Terminology: Satisfiability as a problem (SAT)

- **SAT** is in **NP** as it is easy to verify candidate solutions.
- **SAT** is **NP**-complete (Cook–Levin Theorem), that is, other problems in **NP** reduce to it in polynomial time.
- Moreover, they reduce in a natural way, which is very useful.
- k-CNF: every clause contains at most k literals; the problem: ***k*-SAT**.
  A clause containing k literals: k-clause.
- Even **3-SAT** is already **NP**-complete.
- Sometimes **SAT** is called "CNF SAT". One can consider non-CNF SAT (arbitrary formulas using Boolean operations).

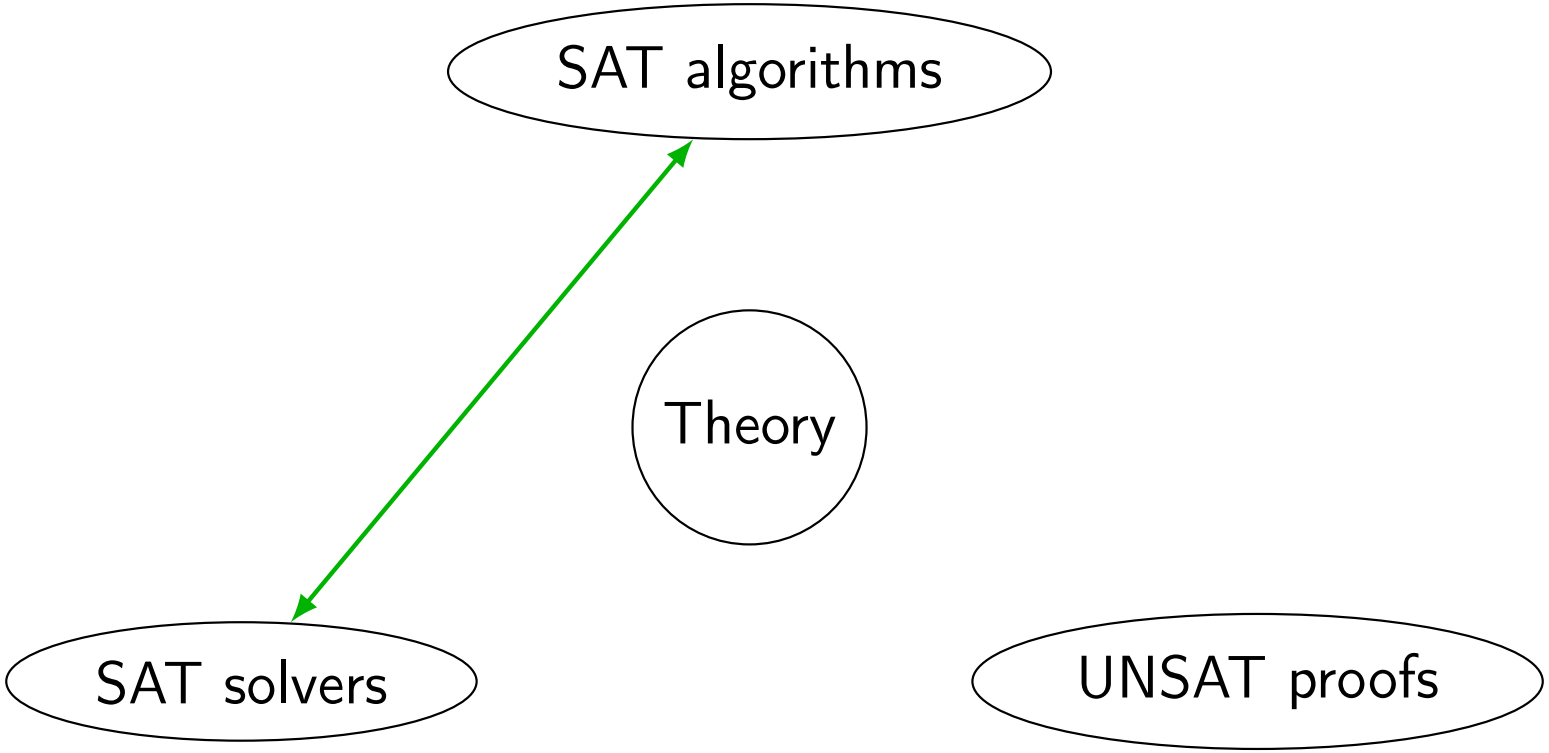
Exercise (Warmup)

*Reduce **3-SAT** to its instances containing at most 3 occurrences of every variable.*

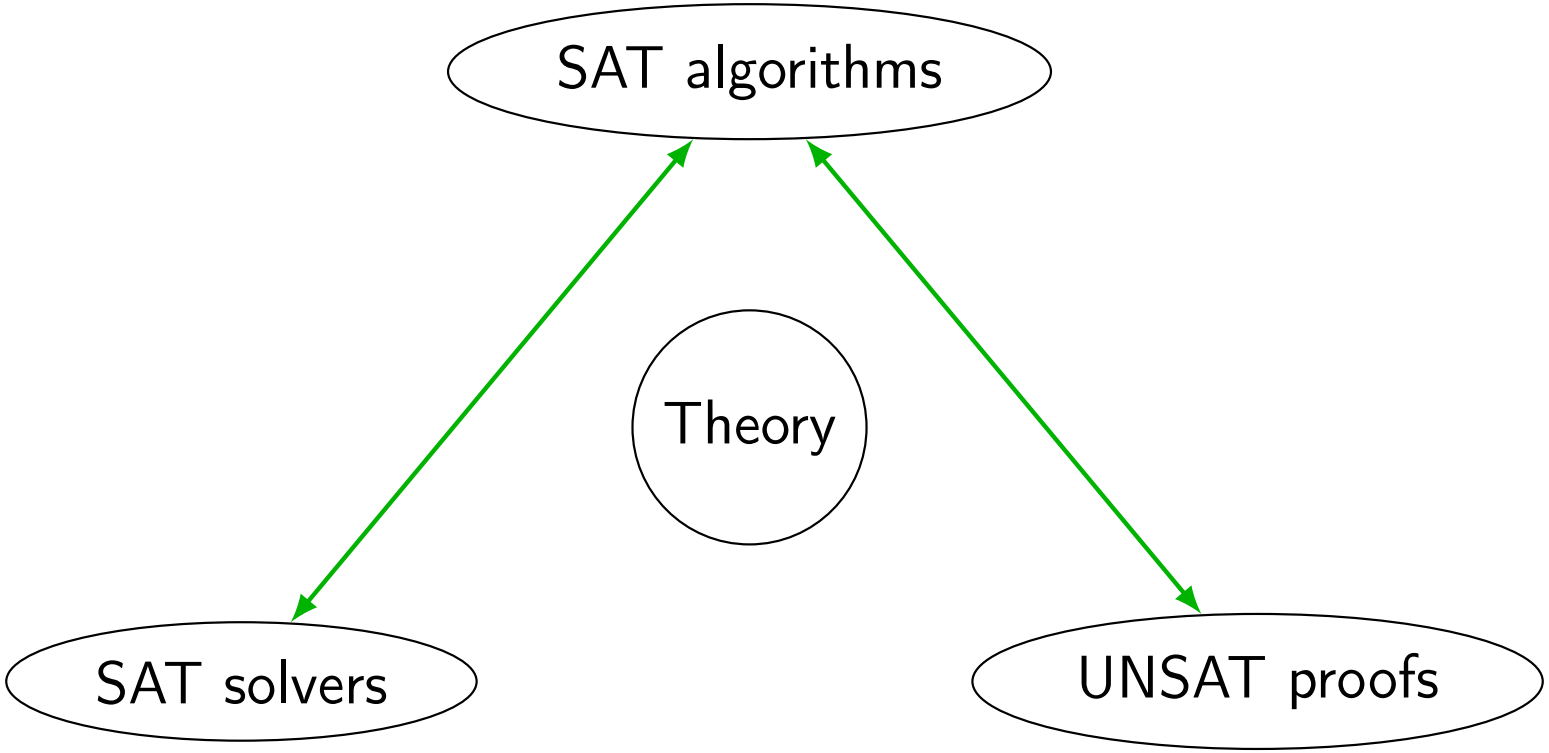
Course overview



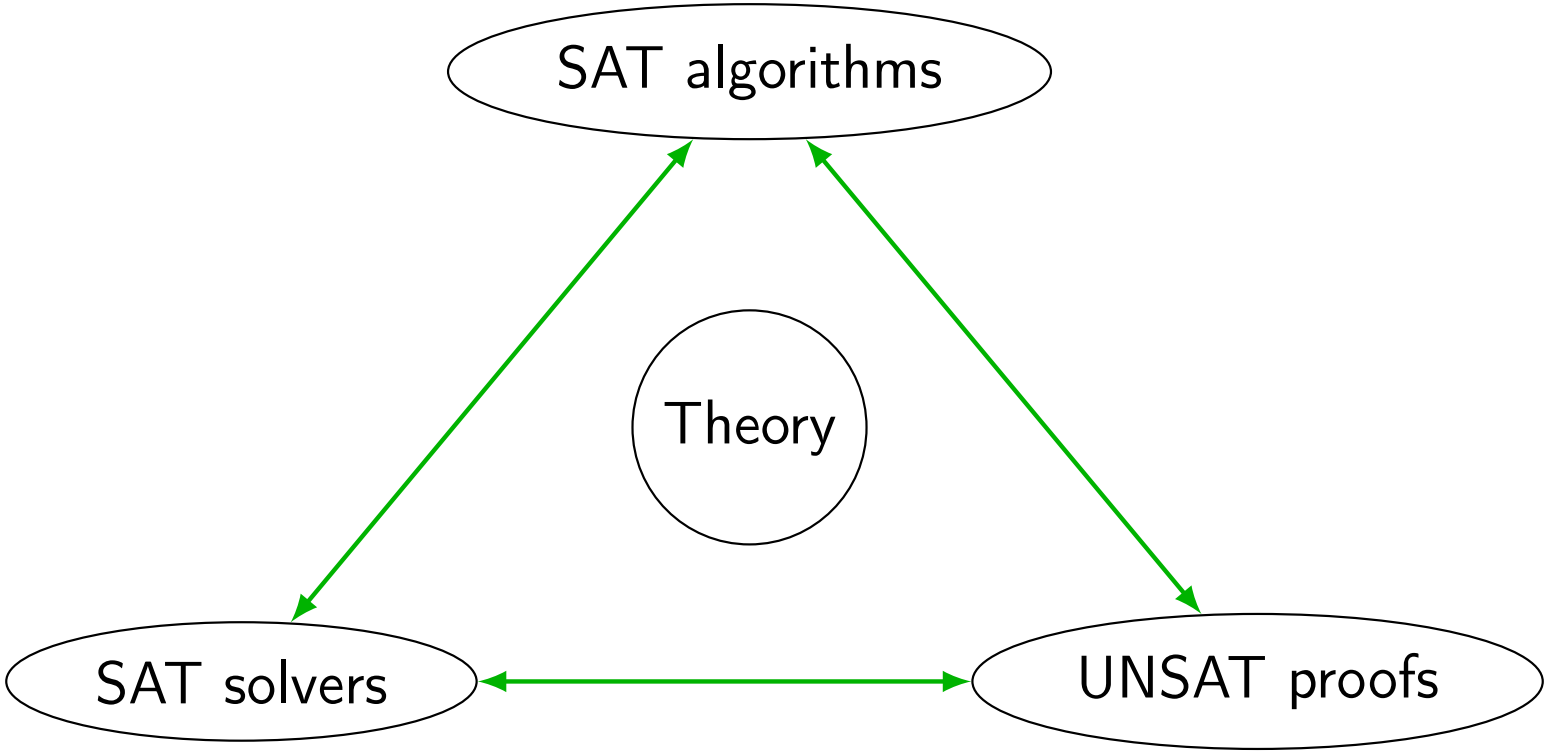
Course overview



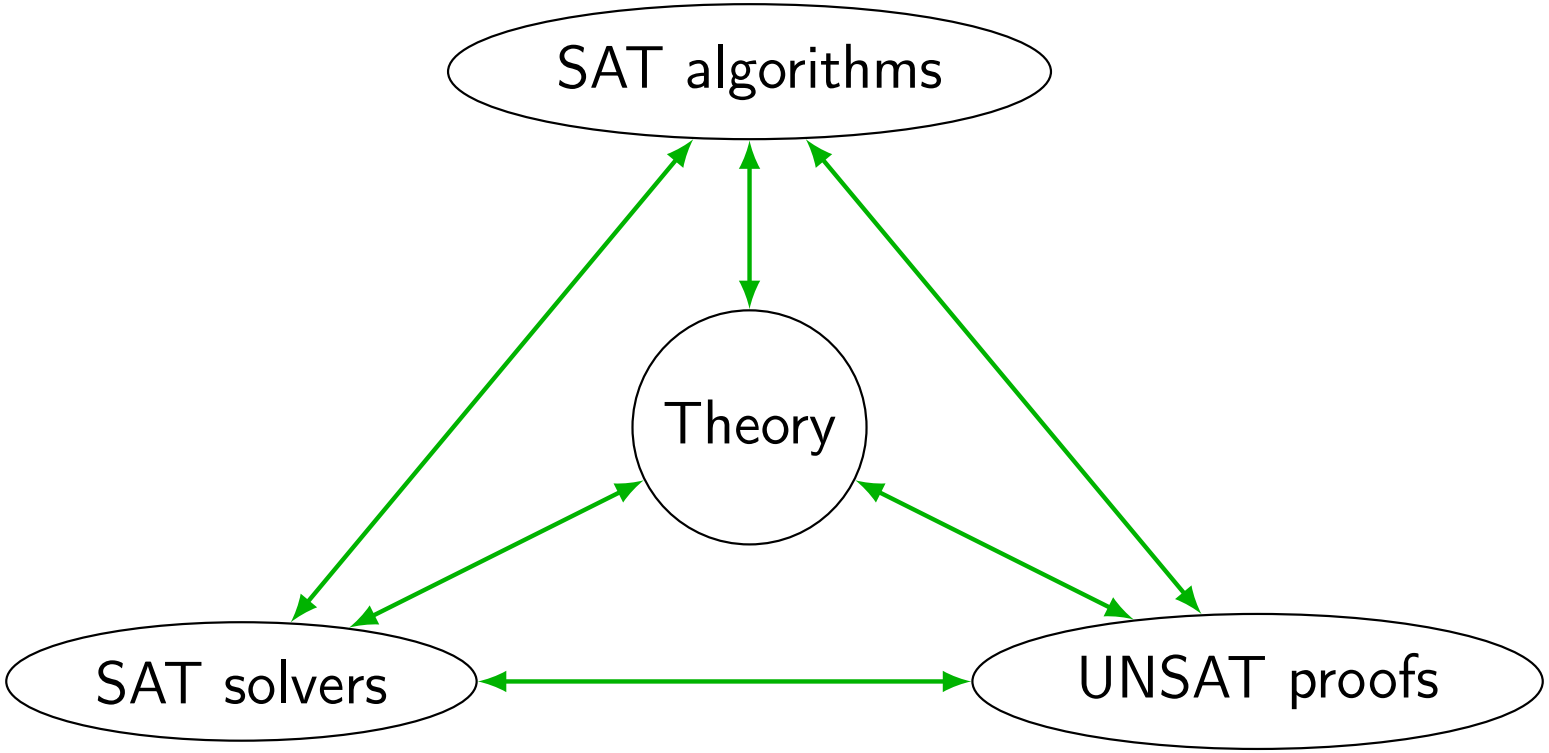
Course overview



Course overview



Course overview



Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.
- Conflict Driven Clause Learning (CDCL). Internals of CDCL solvers.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.
- Conflict Driven Clause Learning (CDCL). Internals of CDCL solvers.
- Resolution width, proof size and proof search. Lower bounds.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.
- Conflict Driven Clause Learning (CDCL). Internals of CDCL solvers.
- Resolution width, proof size and proof search. Lower bounds.
- Proof system as a formal notion. Hierarchy of proof systems.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.
- Conflict Driven Clause Learning (CDCL). Internals of CDCL solvers.
- Resolution width, proof size and proof search. Lower bounds.
- Proof system as a formal notion. Hierarchy of proof systems.
- Integer optimization. Proof systems related to it.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.
- Conflict Driven Clause Learning (CDCL). Internals of CDCL solvers.
- Resolution width, proof size and proof search. Lower bounds.
- Proof system as a formal notion. Hierarchy of proof systems.
- Integer optimization. Proof systems related to it.
- Interpolation and a lower bound for Gomory-Chvátal system.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.
- Conflict Driven Clause Learning (CDCL). Internals of CDCL solvers.
- Resolution width, proof size and proof search. Lower bounds.
- Proof system as a formal notion. Hierarchy of proof systems.
- Integer optimization. Proof systems related to it.
- Interpolation and a lower bound for Gomory-Chvátal system.
- More powerful integer optimization systems.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.
- Conflict Driven Clause Learning (CDCL). Internals of CDCL solvers.
- Resolution width, proof size and proof search. Lower bounds.
- Proof system as a formal notion. Hierarchy of proof systems.
- Integer optimization. Proof systems related to it.
- Interpolation and a lower bound for Gomory-Chvátal system.
- More powerful integer optimization systems.
- Frege and Extended Frege systems.

Course workflow (to be updated!)

- Polynomial-time tractable CNFs. Constraint satisfaction. Dichotomy.
- **Circuit-SAT**. Equivalence checking as a "model" application.
- Practical **SAT** solving (overview).
- Moderately exponential-time algorithms for **SAT** and ***k*-SAT**.
- (Sub)Exponential Time Hypothesis.
- Logical deduction. Resolution.
- Versions of Resolution and their equivalence to **SAT** algorithms.
- Conflict Driven Clause Learning (CDCL). Internals of CDCL solvers.
- Resolution width, proof size and proof search. Lower bounds.
- Proof system as a formal notion. Hierarchy of proof systems.
- Integer optimization. Proof systems related to it.
- Interpolation and a lower bound for Gomory-Chvátal system.
- More powerful integer optimization systems.
- Frege and Extended Frege systems.
- Modern methods: communication complexity.

Now, the course!

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
   - A unit clause (ℓ): contains a single literal.
   - `Unit-clause-elimination`[1]`(`F`):`
     `while (there is` $(\ell) \in F$`) do` $F := F[\ell]$.
   - Does not change satisfiability! The formula is still Horn.

[1] Also called unit propagation.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
   - A unit clause (ℓ): contains a single literal.
   - `Unit-clause-elimination`[1]`(`F`):`
     `while (there is` $(\ell) \in F$`) do` $F := F[\ell]$.
   - Does not change satisfiability! The formula is still Horn.
   - If F becomes empty, it is unsatisfiable.
   - If F contains no unit clauses, it is satisfiable by all-0 assignment.

[1] Also called unit propagation.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
   - A unit clause (ℓ): contains a single literal.
   - `Unit-clause-elimination`[1](F):
     `while (there is` $(\ell) \in F$`) do` $F := F[\ell]$.
   - Does not change satisfiability! The formula is still Horn.
   - If F becomes empty, it is unsatisfiable.
   - If F contains no unit clauses, it is satisfiable by all-0 assignment.

$$(x) \quad (y \vee \overline{z}) \quad (\overline{x} \vee \overline{y}) \quad (\overline{x} \vee z)$$

[1] Also called unit propagation.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
   - A unit clause (ℓ): contains a single literal.
   - `Unit-clause-elimination`[1]`(`F`):`
     `while (there is` $(\ell) \in F$`) do` $F := F[\ell]$.
   - Does not change satisfiability! The formula is still Horn.
   - If F becomes empty, it is unsatisfiable.
   - If F contains no unit clauses, it is satisfiable by all-0 assignment.

$$(x) \quad (y \vee \overline{z}) \quad (\overline{x} \vee \overline{y}) \quad (\overline{x} \vee z)$$

$$\boxed{(x)} \quad (y \vee \overline{z}) \quad (\overline{x} \vee \overline{y}) \quad (\overline{x} \vee z)$$

[1] Also called unit propagation.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
   - A unit clause (ℓ): contains a single literal.
   - `Unit-clause-elimination`[1](F):
     `while (there is` $(\ell) \in F$`) do` $F := F[\ell]$.
   - Does not change satisfiability! The formula is still Horn.
   - If F becomes empty, it is unsatisfiable.
   - If F contains no unit clauses, it is satisfiable by all-0 assignment.

$$
\begin{array}{cccc}
(x) & (y \vee \overline{z}) & (\overline{x} \vee \overline{y}) & (\overline{x} \vee z) \\
\boxed{(x)} & (y \vee \overline{z}) & (\overline{x} \vee \overline{y}) & (\overline{x} \vee z) \\
 & (y \vee \overline{z}) & (\overline{y}) & (z)
\end{array}
$$

[1] Also called unit propagation.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
   - A unit clause (ℓ): contains a single literal.
   - `Unit-clause-elimination`[1]`(`F`):`
     `while (there is` $(\ell) \in F$`) do` $F := F[\ell]$.
   - Does not change satisfiability! The formula is still Horn.
   - If F becomes empty, it is unsatisfiable.
   - If F contains no unit clauses, it is satisfiable by all-0 assignment.

$$
\begin{array}{cccc}
(x) & (y \vee \overline{z}) & (\overline{x} \vee \overline{y}) & (\overline{x} \vee z) \\
\boxed{(x)} & (y \vee \overline{z}) & (\overline{x} \vee \overline{y}) & (\overline{x} \vee z) \\
 & (y \vee \overline{z}) & (\overline{y}) & (z) \\
 & (y \vee \overline{z}) & (\overline{y}) & \boxed{(z)}
\end{array}
$$

[1] Also called unit propagation.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
   - A unit clause (ℓ): contains a single literal.
   - `Unit-clause-elimination`[1](F):
     `while (there is` $(\ell) \in F$`) do` $F := F[\ell]$.
   - Does not change satisfiability! The formula is still Horn.
   - If F becomes empty, it is unsatisfiable.
   - If F contains no unit clauses, it is satisfiable by all-0 assignment.

(x)	$(y \vee \overline{z})$	$(\overline{x} \vee \overline{y})$	$(\overline{x} \vee z)$
$\boxed{(x)}$	$(y \vee \overline{z})$	$(\overline{x} \vee \overline{y})$	$(\overline{x} \vee z)$
	$(y \vee \overline{z})$	$(\overline{y})$	(z)
	$(y \vee \overline{z})$	$(\overline{y})$	$\boxed{(z)}$
	(y)	$(\overline{y})$	

[1] Also called unit propagation.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
   - A unit clause (ℓ): contains a single literal.
   - `Unit-clause-elimination`[1](F):
     `while (there is` $(\ell) \in F$`) do` $F := F[\ell]$.
   - Does not change satisfiability! The formula is still Horn.
   - If F becomes empty, it is unsatisfiable.
   - If F contains no unit clauses, it is satisfiable by all-0 assignment.

(x)	$(y \vee \overline{z})$	$(\overline{x} \vee \overline{y})$	$(\overline{x} \vee z)$
$\boxed{(x)}$	$(y \vee \overline{z})$	$(\overline{x} \vee \overline{y})$	$(\overline{x} \vee z)$
	$(y \vee \overline{z})$	$(\overline{y})$	(z)
	$(y \vee \overline{z})$	$(\overline{y})$	$\boxed{(z)}$
	(y)	$(\overline{y})$	
	$\boxed{(y)}$	$(\overline{y})$	

It is extremely important in practice to optimize this procedure!

[1] Also called unit propagation.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
2. Anti-Horn[1] formulas: at most one negative literal per clause.

[1] Also called dual-Horn.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
2. Anti-Horn[1] formulas: at most one negative literal per clause.
3. Systems of affine[2] equations modulo two[3]. (By Gaussian elimination.) Every equation in k variables is expressed by 2^{k-1} clauses that can be easily recognized.

$$\begin{aligned} x_1 \oplus x_2 \oplus x_3 &= 1 \\ x_2 \oplus x_4 \oplus x_5 &= 0 \\ x_1 \oplus x_4 &= 0 \\ x_1 \oplus x_5 &= 1 \end{aligned}$$

$$x \oplus y \qquad \sim \qquad (x \vee y) \wedge (\overline{x} \vee \overline{y})$$

[1] Also called dual-Horn.

[2] Linear combination = constant.

[3] That is, xor-relations $x_{i_1} \oplus \ldots \oplus x_{i_k} = 0$ or $= 1$.

Polynomial-time tractable classes of formulas

1. Horn formulas: every clause contains at most one positive literal.
2. Anti-Horn[1] formulas: at most one negative literal per clause.
3. Systems of affine[2] equations modulo two. (By Gaussian elimination.) Every equation in k variables is expressed by 2^{k-1} clauses that can be easily recognized.
4. **2-SAT**.

[1] Also called dual-Horn.

[2] Linear combination = constant.

2-SAT

A reminder from Algorithms-2

- Formula $\mapsto$ directed graph.
- Literals $\mapsto$ vertices.
- Clause $\ell \vee \ell'$ is mapped to edges $\overline{\ell} \to \ell'$ and $\overline{\ell'} \to \ell$.
- Formula is unsatifiable $\iff$
  for some x, there is a directed cycle including x and $\overline{x}$.

Algorithm: Construct strongly connected components and check this.

Note: the optimization problem **MAX-SAT** asking for an assignment satisfying the maximum possible number of clauses, remains **NP**-hard even for 2-CNFs.

Constraint Satisfaction Problem

Examples

CNF SAT is a particular case of Constraint Satisfaction Problem (**CSP**).

Asks for a satisfying assignment to a conjunction of relation(s)-based conditions.

	SAT	**3-COLORING**	**CSP**
variables	{0,1}	{0,1,2}	any
conditions	disjunctions of literals	non-equality	some relations

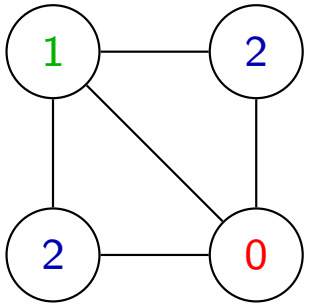
Example (3-COLORING): Vertex coloring of a graph in 3 colors.

Domain: Three colors $\{0, 1, 2\}$.

Relation: $R(x, y) =$ "$\boldsymbol{x \neq y}$".

Input:

$$\{(R, \{x_u, x_v\}) : \{u, v\} \in E\}$$



for a graph $G = (V, E)$ and variables x_v for $v \in V$.

Output: $f : V \to \{0, 1, 2\}$ s.t. $f(x) \neq f(y)$ for every edge $\{u, v\} \in E$.

Constraint Satisfaction Problem

Examples

CNF SAT is a particular case of Constraint Satisfaction Problem (**CSP**).

Asks for a satisfying assignment to a conjunction of relation(s)-based conditions.

	SAT	**3-COLORING**	**CSP**
variables	{0,1}	{0,1,2}	any
conditions	disjunctions of literals	non-equality	some relations

Example (2-SAT):

Domain: Boolean $\{\texttt{False}, \texttt{True}\}$.

Relations:

$$R_1(x,y) = x \vee y \qquad R_5(x) = x$$
$$R_2(x,y) = x \vee \overline{y} \qquad R_6(x) = \overline{x}$$
$$R_3(x,y) = \overline{x} \vee y$$
$$R_4(x,y) = \overline{x} \vee \overline{y}$$

Example of a formula: $R_1(x,y) \wedge R_2(x,z) \wedge R_6(x)$.

Constraint Satisfaction Problem

Formal definition

CSP: parameterized by a finite domain $D = [1\,..\,n]$ and a set of relations

$$R_i : D^{a_i} \to \{0, 1\}.$$

Input: A set of constraints of the form

$$C_j = (r_j, s_j),$$

where

r_j is one of the R_i's,
$d_j = a_i$,
s_j is a d_j-size tuple of D-valued variables $\{x_1, \ldots, x_n\}$.

Output: a satisfying assignment $f : [1\,..\,n] \to D$:

$$r_j(f(s_{j,1}), \ldots, f(s_{j,d_j})) = 1$$

or "no" if it does not exist.

Schaefer's dichotomy theorem

0. CSP over $\{0, 1\}$ for clauses (disjunctions where some arguments are negated) is called **SAT**.

Schaefer's dichotomy theorem

0. CSP over $\{0, 1\}$ for clauses (disjunctions where some arguments are negated) is called **SAT**.

Tractable cases:

1. CSP for 2-clauses and 1-clauses (and their conjunctions) is called **2-SAT**.
2. CSP for xor-relations $x_{i_1} \oplus \ldots \oplus x_{i_k} = \mathrm{const}$ (and their conjunctions) is called **XOR-SAT**.
3. CSP for Horn clauses (and their conjunctions) is called **Horn-SAT**.
4. CSP for anti-Horn clauses (and their conjunctions) is called **anti-Horn-SAT**.
5. CSP for all relations R such that $R(1, \ldots, 1) = 1$ (called 1-valid).
6. CSP for all relations R such that $R(0, \ldots, 0) = 1$ (called 0-valid).

Schaefer's dichotomy theorem

0. CSP over $\{0, 1\}$ for clauses (disjunctions where some arguments are negated) is called **SAT**.

Tractable cases:

1. CSP for 2-clauses and 1-clauses (and their conjunctions) is called **2-SAT**.
2. CSP for xor-relations $x_{i_1} \oplus \ldots \oplus x_{i_k} = \mathrm{const}$ (and their conjunctions) is called **XOR-SAT**.
3. CSP for Horn clauses (and their conjunctions) is called **Horn-SAT**.
4. CSP for anti-Horn clauses (and their conjunctions) is called **anti-Horn-SAT**.
5. CSP for all relations R such that $R(1, \ldots, 1) = 1$ (called 1-valid).
6. CSP for all relations R such that $R(0, \ldots, 0) = 1$ (called 0-valid).

Theorem (Schaefer, 1978)

*Consider a set of relations $\mathcal{R}$ over $\{0, 1\}$. If all of them belong to the same single type out of (1)–(6), plus trivially false and trivially true relations, then **CSP** for $\mathcal{R}$ is polynomial-time solvable.*

*Otherwise it is **NP**-complete.*

Schaefer's dichotomy theorem

0. CSP over $\{0,1\}$ for clauses (disjunctions where some arguments are negated) is called **SAT**.

Tractable cases:

1. CSP for 2-clauses and 1-clauses (and their conjunctions) is called **2-SAT**.
2. CSP for xor-relations $x_{i_1} \oplus \ldots \oplus x_{i_k} = \mathrm{const}$ (and their conjunctions) is called **XOR-SAT**.
3. CSP for Horn clauses (and their conjunctions) is called **Horn-SAT**.
4. CSP for anti-Horn clauses (and their conjunctions) is called **anti-Horn-SAT**.
5. CSP for all relations R such that $R(1,\ldots,1) = 1$ (called 1-valid).
6. CSP for all relations R such that $R(0,\ldots,0) = 1$ (called 0-valid).

Theorem (Schaefer, 1978)

*Consider a set of relations $\mathcal{R}$ over $\{0,1\}$. If all of them belong to the same single type out of (1)–(6), plus trivially false and trivially true relations, then **CSP** for $\mathcal{R}$ is polynomial-time solvable.* $\longleftarrow$ *[We have already proved it.]*

*Otherwise it is **NP**-complete.* $\longleftarrow$ *[We won't prove it.]*

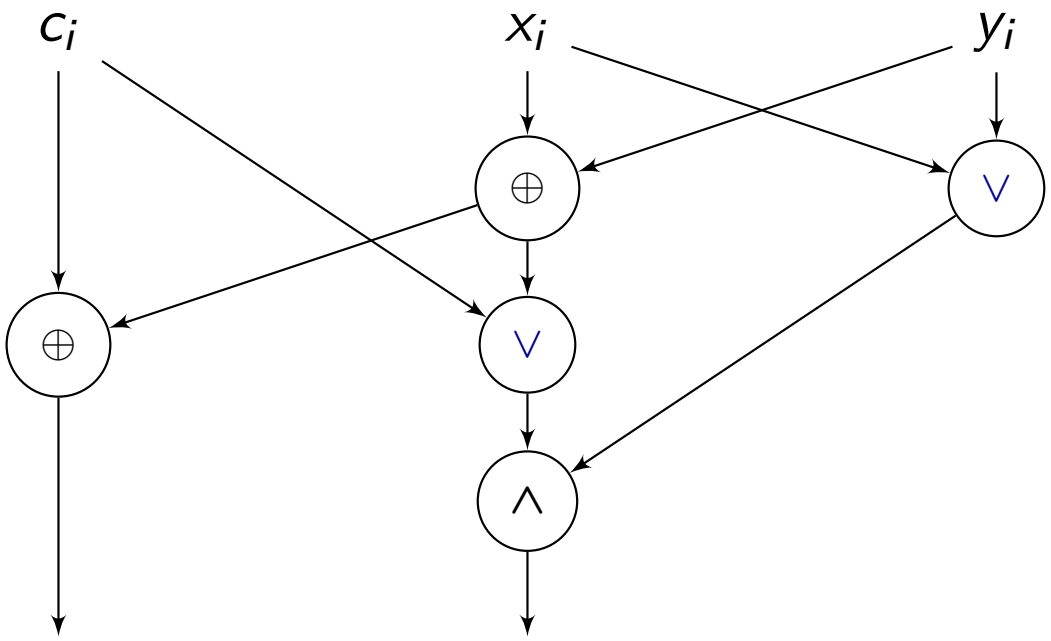
Takeaway message:

2-SAT, (Anti)Horn, **XOR-SAT** are **P**-time solvable,
unless you mix them!

Boolean circuits

- A directed acyclic graph.
- Binary Boolean operations in the nodes.
- Sources (in-degree zero): inputs, or variables.
- Outputs: designated nodes.
- Formalizes computation on fixed-length inputs (think about CPU).

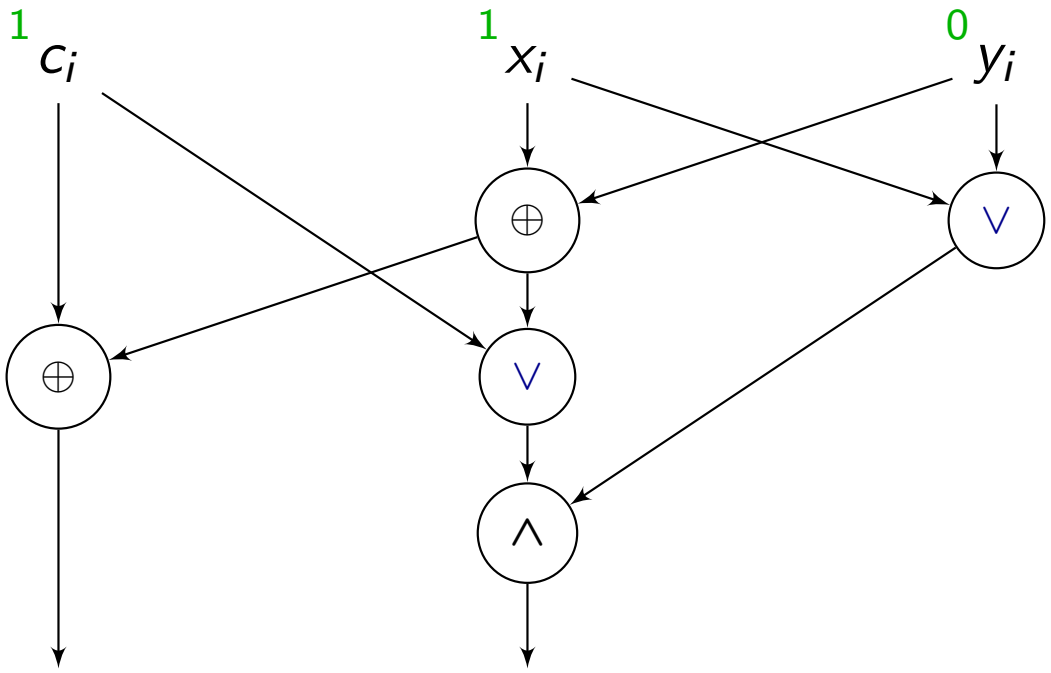
Example: **Full Adder**.



Boolean circuits

- A directed acyclic graph.
- Binary Boolean operations in the nodes.
- Sources (in-degree zero): inputs, or variables.
- Outputs: designated nodes.
- Formalizes computation on fixed-length inputs (think about CPU).

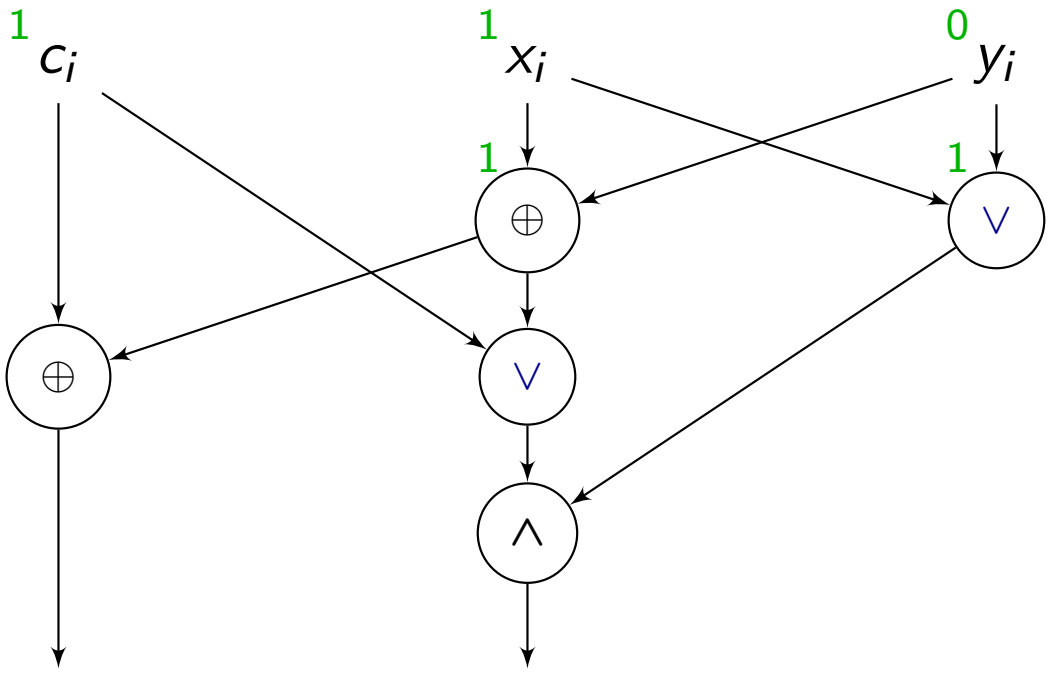
Example: **Full Adder**.



Boolean circuits

- A directed acyclic graph.
- Binary Boolean operations in the nodes.
- Sources (in-degree zero): inputs, or variables.
- Outputs: designated nodes.
- Formalizes computation on fixed-length inputs (think about CPU).

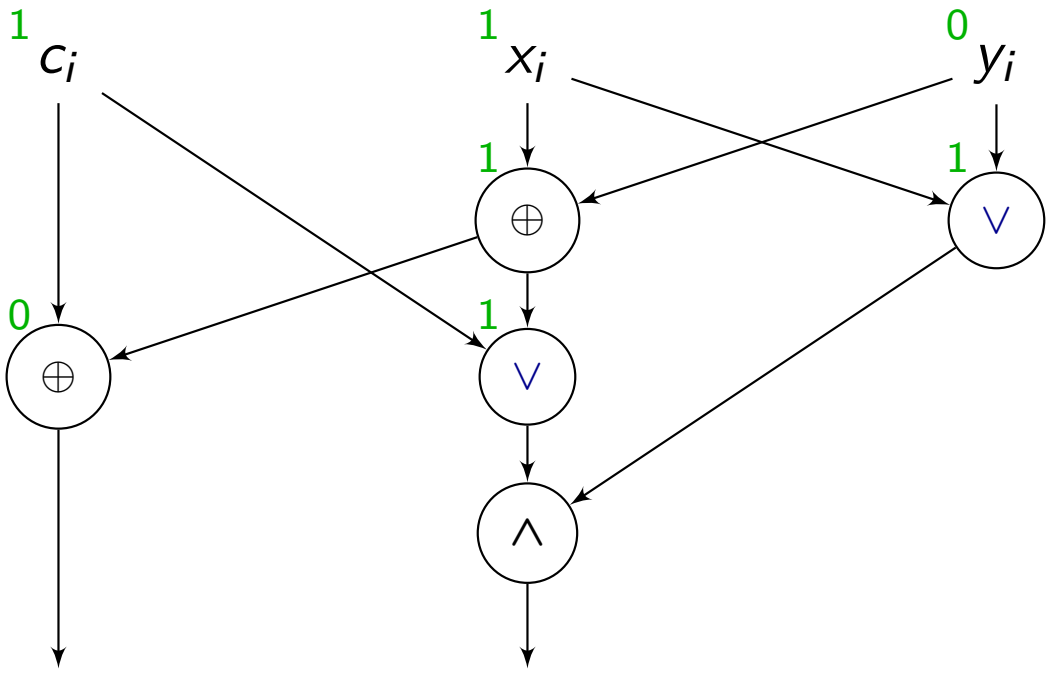
Example: **Full Adder**.



Boolean circuits

- A directed acyclic graph.
- Binary Boolean operations in the nodes.
- Sources (in-degree zero): inputs, or variables.
- Outputs: designated nodes.
- Formalizes computation on fixed-length inputs (think about CPU).

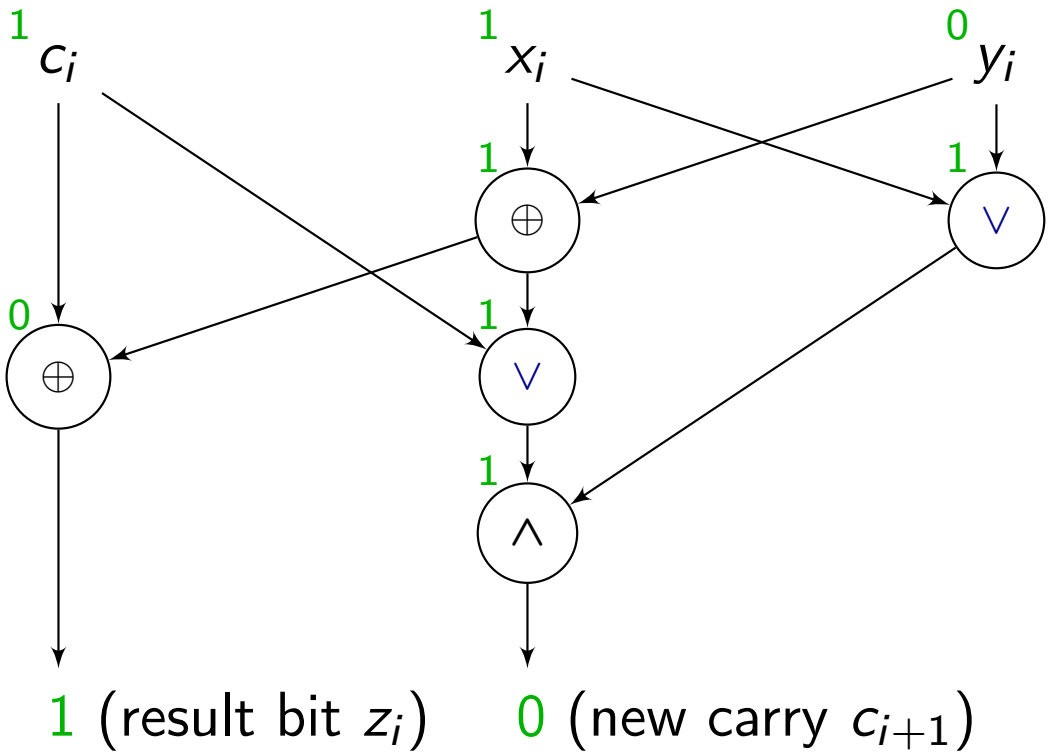
Example: **Full Adder**.



Boolean circuits

- A directed acyclic graph.
- Binary Boolean operations in the nodes.
- Sources (in-degree zero): inputs, or variables.
- Outputs: designated nodes.
- Formalizes computation on fixed-length inputs (think about CPU).

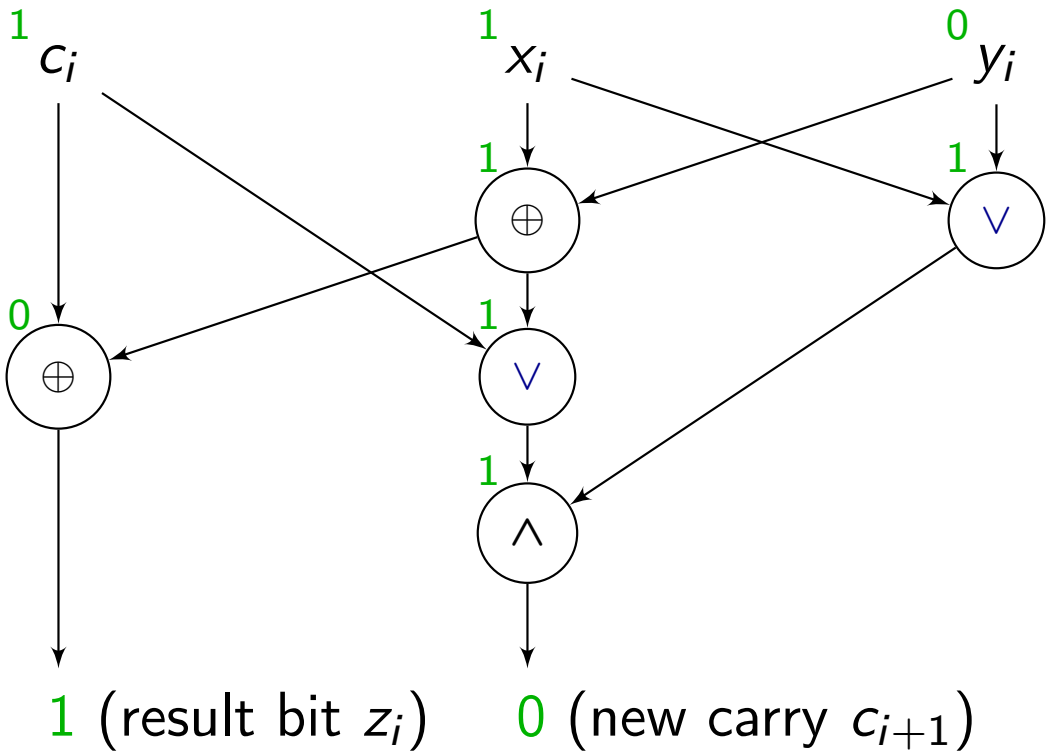
Example: **Full Adder**.



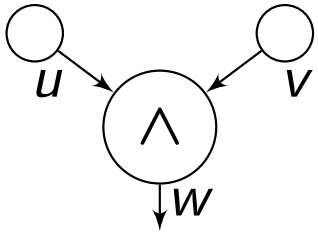
Boolean circuits

- A directed acyclic graph.
- Binary Boolean operations in the nodes.
- Sources (in-degree zero): inputs, or variables.
- Outputs: designated nodes.
- Formalizes computation on fixed-length inputs (think about CPU).

Example: **Full Adder**.



Tseitin's translation to CNFs:



$(\overline{w} \vee u) \quad \wedge$

$(\overline{w} \vee v) \quad \wedge$

$(\overline{u} \vee \overline{v} \vee w)$

Thus **Circuit-SAT** is polynomially equivalent to **SAT**.

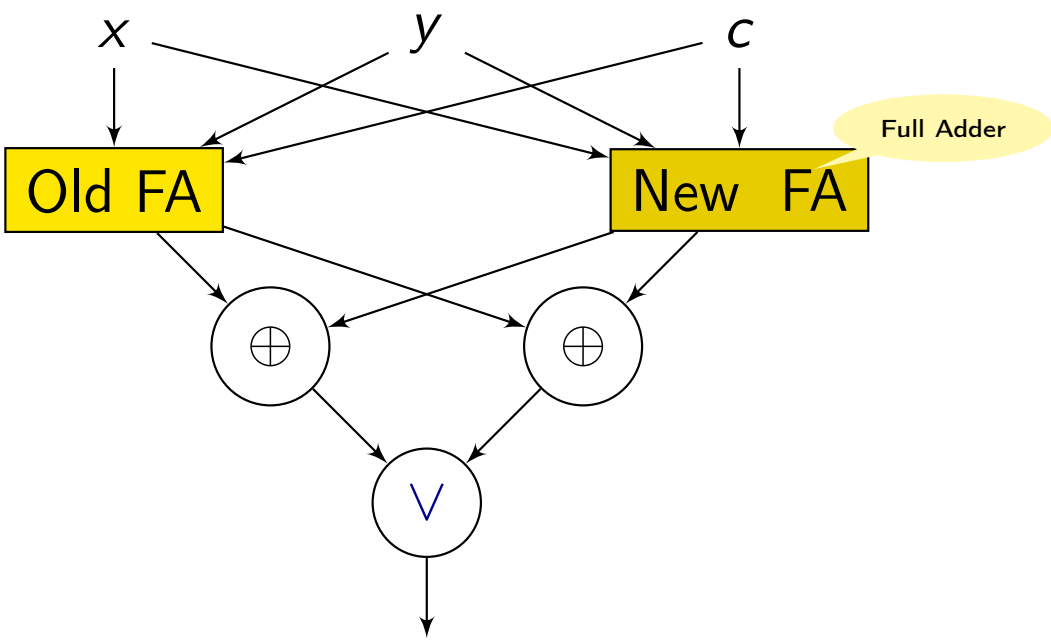
k-CNF to 3-CNF

Exercise (k-SAT$\rightarrow$3-SAT)

*Reduce **k-SAT** to **3-SAT*** without *introducing new variables.*
Then show how to restore the satisfying assignment.

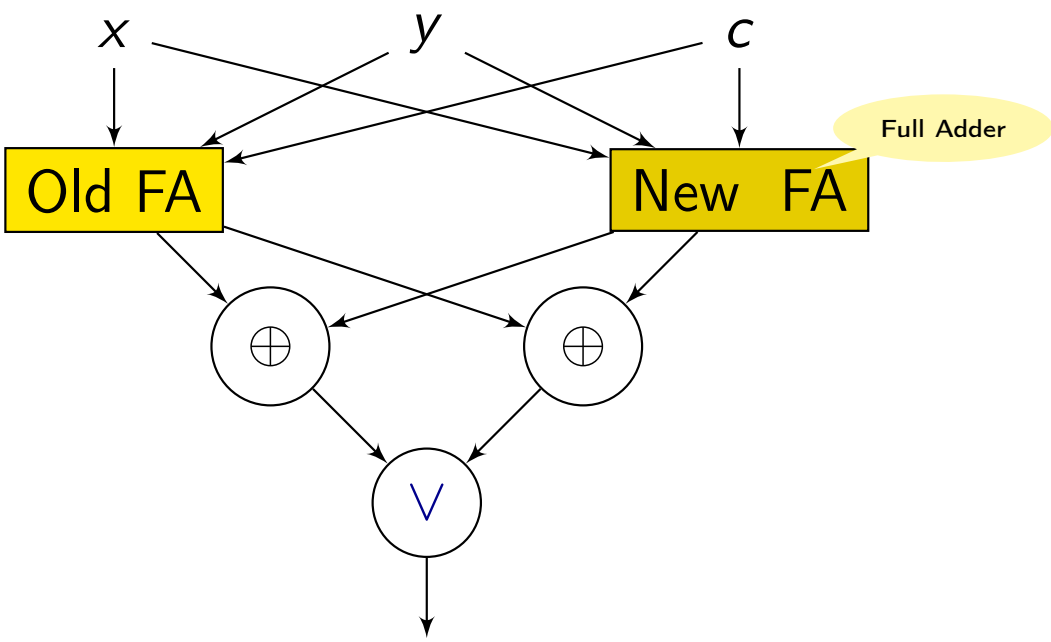
Equivalence checking

- $\exists ? x_1, \ldots, x_n$ $f(x_1 \cdots x_n) \neq g(x_1 \cdots x_n)$.
- Expressed as a circuit comparing the outputs of f and g.
- Equivalent to **SAT**.
- A major problem in circuit design and verification.



Equivalence checking

- $\exists ? x_1, \ldots, x_n$ $f(x_1 \cdots x_n) \neq g(x_1 \cdots x_n)$.
- Expressed as a circuit comparing the outputs of f and g.
- Equivalent to **SAT**.
- A major problem in circuit design and verification.



- Found a satisfying assignment ⇒ found a bug in CPU!
- If not found . . . can you prove it?
- Companies strongly prefer to have provably correct hardware.

Practical SAT solving

- SAT solver: a program that solves **SAT**.

[3] www.satcompetition.org

[4] drops.dagstuhl.de/opus/volltexte/2022/16682/pdf/LIPIcs-SAT-2022-8.pdf

Practical SAT solving

- SAT solver: a program that solves **SAT**.
- How come? **SAT** is **NP**-complete!

[3] www.satcompetition.org

[4] drops.dagstuhl.de/opus/volltexte/2022/16682/pdf/LIPIcs-SAT-2022-8.pdf

Practical SAT solving

- SAT solver: a program that solves **SAT**.
- How come? **SAT** is **NP**-complete!
- Uses heuristics, and is very slow on some inputs.
- Industrial benchmarks are frequently "easy"
  (for example, contain lots of 2-clauses).
- Thus data structures and efficient implementation are very important.

[3] www.satcompetition.org

[4] drops.dagstuhl.de/opus/volltexte/2022/16682/pdf/LIPIcs-SAT-2022-8.pdf

Practical SAT solving

- SAT solver: a program that solves **SAT**.
- How come? **SAT** is **NP**-complete!
- Uses heuristics, and is very slow on some inputs.
- Industrial benchmarks are frequently "easy"
  (for example, contain lots of 2-clauses).
- Thus data structures and efficient implementation are very important.
- Not only industry: need to check your combinatorial conjecture?
  Write down a CNF and run a SAT solver!

[3] www.satcompetition.org

[4] drops.dagstuhl.de/opus/volltexte/2022/16682/pdf/LIPIcs-SAT-2022-8.pdf

Practical SAT solving

- SAT solver: a program that solves **SAT**.
- How come? **SAT** is **NP**-complete!
- Uses heuristics, and is very slow on some inputs.
- Industrial benchmarks are frequently "easy"
  (for example, contain lots of 2-clauses).
- Thus data structures and efficient implementation are very important.
- Not only industry: need to check your combinatorial conjecture?
  Write down a CNF and run a SAT solver!
- SAT Competitions[3]: a regular race of SAT solvers.
- Everyone writes a SAT solver including Intel-Haifa (Alexander Nadel)[4].

[3] www.satcompetition.org

[4] drops.dagstuhl.de/opus/volltexte/2022/16682/pdf/LIPIcs-SAT-2022-8.pdf

Practical SAT solving

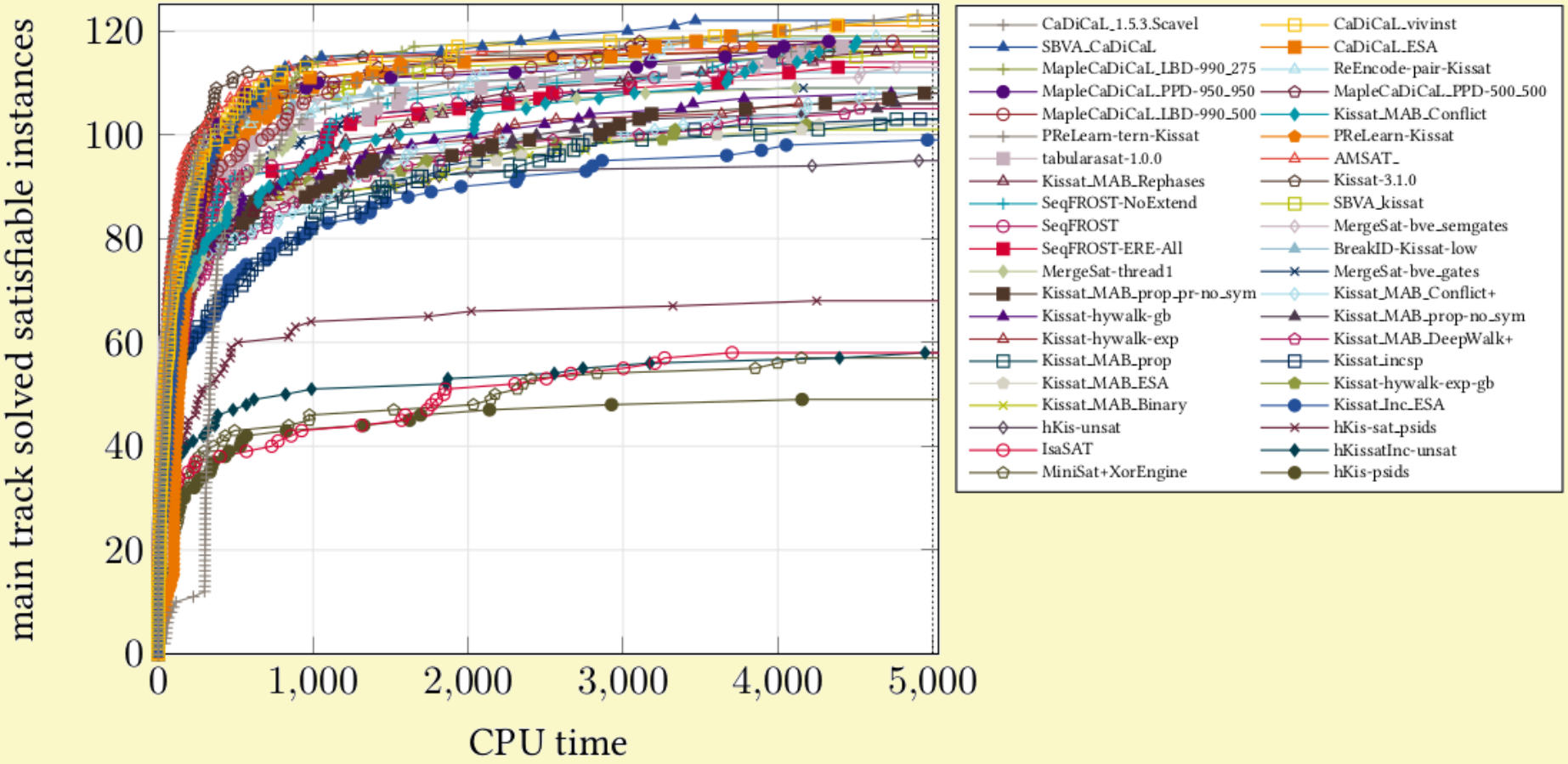
- SAT solver: a program that solves **SAT**.
- How come? **SAT** is **NP**-complete!
- Uses heuristics, and is very slow on some inputs.
- Industrial benchmarks are frequently "easy" (for example, contain lots of 2-clauses).
- Thus data structures and efficient implementation are very important.
- Not only industry: need to check your combinatorial conjecture? Write down a CNF and run a SAT solver!
- SAT Competitions[3]: a regular race of SAT solvers.
- Everyone writes a SAT solver including Intel-Haifa (Alexander Nadel)[4].
- SAT Competitions maintain a database of benchmarks
  - Industrial (equivalence checking, and more).
  - Handcrafted (known to be hard: PHP etc).
  - Randomly generated.

[3] www.satcompetition.org

[4] drops.dagstuhl.de/opus/volltexte/2022/16682/pdf/LIPIcs-SAT-2022-8.pdf

SAT Competition[5]

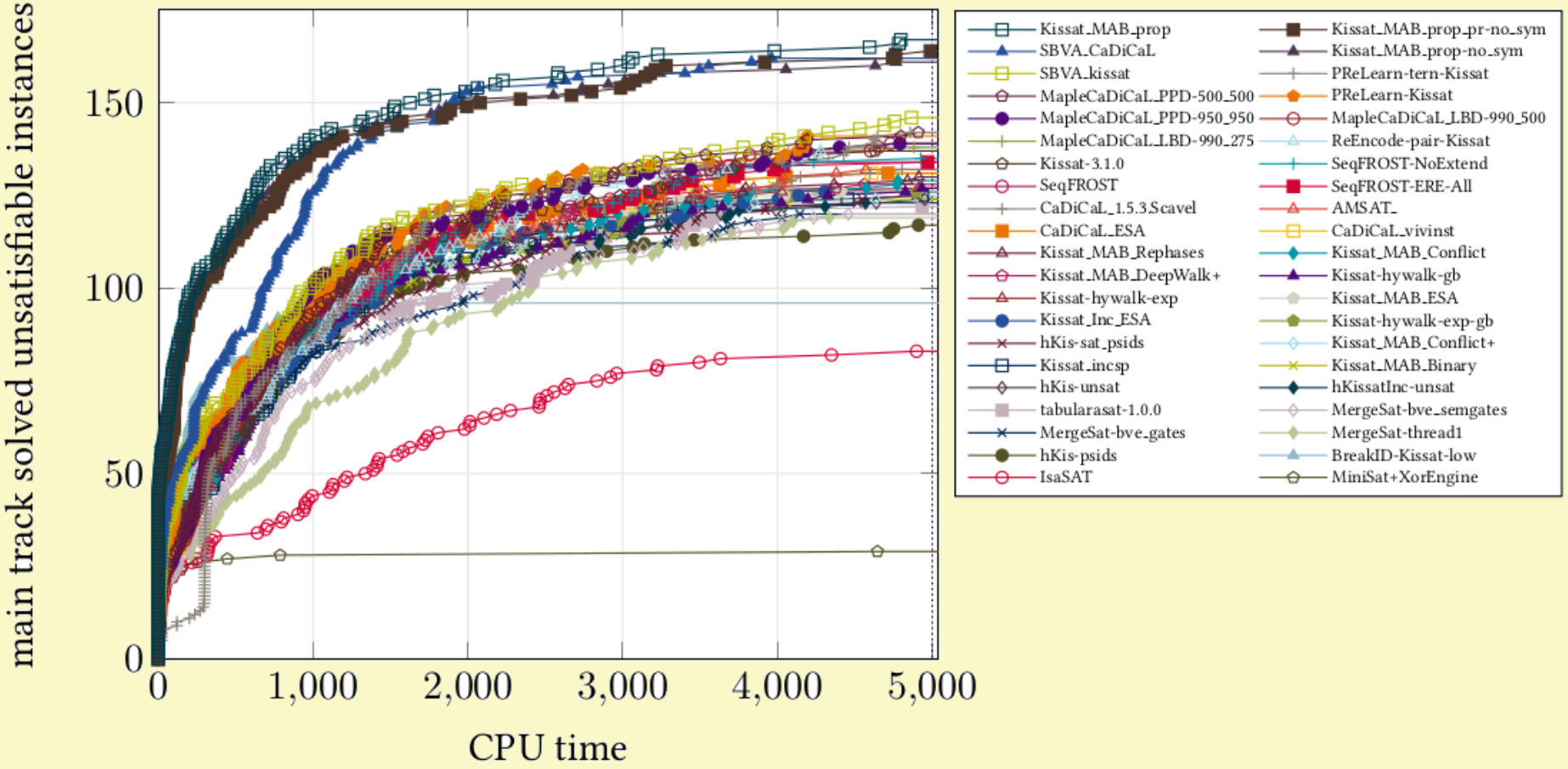
Main (Sequential) Track SAT Plot



[5] https://satcompetition.github.io/2023/downloads/satcomp23slides.pdf

SAT Competition[5]

Main (Sequential) Track UNSAT Plot



[5] https://satcompetition.github.io/2023/downloads/satcomp23slides.pdf

SAT Solvers

Classification

- Complete:
  - outputs a satisfying assignment, if it finds one,
  - says "UNSAT" if there is none,
    - may produce a proof of unsatisfiability.
- Incomplete:
  - can also return "UNKNOWN".

SAT Solvers

Classification

- Complete:
  - outputs a satisfying assignment, if it finds one,
  - says "UNSAT" if there is none,
    - may produce a proof of unsatisfiability.
- Incomplete:
  - can also return "UNKNOWN".

- Randomized (sometimes also called incomplete. . .),
- Deterministic.

SAT Solvers

Classification

- Complete:
  - outputs a satisfying assignment, if it finds one,
  - says "UNSAT" if there is none,
    - may produce a proof of unsatisfiability.
- Incomplete:
  - can also return "UNKNOWN".

- Randomized (sometimes also called incomplete. . .),
- Deterministic.

- Parallel.
- Sequential.

Input and Output

Input

- Benchmarks.
  These are CNFs on which the algorithms compete.
  - (Simplified) DIMACS format

```
c
c start with comments
c here is a CNF with 5 variables and 3 clauses
c
p cnf 5 3
1 -5 4 0
-1 5 3 4 0
-3 -4 0
```

$$(x_1 \vee \overline{x}_5 \vee x_4) \wedge (\overline{x}_1 \vee x_5 \vee x_3 \vee x_4) \wedge (\overline{x}_3 \vee \overline{x}_4)$$

  - RTL format
    - Serves for describing Boolean circuits

Input and Output

Output

- Assignments

```
c Comments are still allowed
s SATISFIABLE
v 1 3
v -4 6 0
```

$x_1, x_3, \overline{x}_4, x_6$

Input and Output

Output

- Assignments

```
c Comments are still allowed
s SATISFIABLE
v 1 3
v -4 6 0
```

$x_1, x_3, \overline{x}_4, x_6$

- Proofs of unsatisfiability (if required)
  - DRAT format (too early to explain)

- Format descriptions and other rules:
  https://satcompetition.github.io/2023/

Takeaway (summary)

- We set up the **terminology** for this course.
- **SAT** is central to TCS.
- **Search-to-decision** reduction.
- Circuit **equivalence checking** as a "model" application.
- It is important both to **find** solutions and to **prove** that there are no solutions.
- "**Easy**" classes of formulas in terms of **CSP** and how to solve them.
- Unit clause elimination (**unit propagation**).
- Schaefer's **dichotomy** theorem (without proof of **NP**-completeness).
- **SAT Competition**.

- Questions? edward.a.hirsch@gmail.com

Takeaway (summary)

- We set up the **terminology** for this course.
- **SAT** is central to TCS.
- **Search-to-decision** reduction.
- Circuit **equivalence checking** as a "model" application.
- It is important both to **find** solutions and to **prove** that there are no solutions.
- "**Easy**" classes of formulas in terms of **CSP** and how to solve them.
- Unit clause elimination (**unit propagation**).
- Schaefer's **dichotomy** theorem (without proof of **NP**-completeness).
- **SAT Competition**.

- Questions? edward.a.hirsch@gmail.com

- *Coming next: moderately exponential time **SAT** algorithms.*

This slide is intentially left empty.